

My contributions to the TML4 Pipeline-Zijiang

I worked on this project twice, months apart. The first time I drew the lines between the modules. The second time I found out whether those lines were real — by bolting an entirely different kind of data onto the far end of the pipeline without touching a single module in it.

Contracts are the cheapest thing to write and the most expensive thing to retrofit. The work on the first branch that mattered wasn't any of the code — it was deciding that each stage should say out loud what it hands to the next. Two months later, that decision was the reason a completely different kind of data could be added by writing new files and changing none.

Branches `zz` and `zz_imaging` · June – August 2026

0 PIPELINE MODULES EDITED FOR IMAGING	102 NAMED FEATURES PER PATIENT	175 TESTS ADDED (71 → 246)	4,424 LINES IN THE IMAGING BRANCH
--	--------------------------------------	----------------------------------	---

zz

ffd667f
19 Jun 2026
“mvp version”

Six modules, and a dataclass at every seam

The project answers one yes/no question: will a patient's cancer respond to a given immunotherapy? Four of us attacked it on separate branches, and mine started from a flat pile of scripts — `m1_loader.py` through `m6_report.py` plus a `pipeline.py` that wired them together and an `app.py` that had grown to 647 lines.

It ran. But nothing declared what it handed to the next thing, so every change meant reading the whole chain to find out what would break. My commit replaced that with six numbered modules, and gave each one an explicit **standardized output** — a dataclass sitting at the top of the file, declaring what leaves it:

MODULE	DECLARES	CARRYING
module1_gui	PipelineConfig	every user selection, in one object
module2_ingestion	ProcessedData	X, y, feature_names
module3_mapping	MappedData	reduced matrix + the mapping rule that made it
module4_training	TrainedTestingPipeline	the fitted machine and how to feed it
module5_prediction	PredictionResults	predictions, patient ids, truth when known
module6_evaluation	EvaluationReport	metrics, confusion matrix, PDF

`module6` was the biggest at 255 lines: accuracy, sensitivity and specificity computed from the raw confusion counts rather than trusted to a one-liner, a 2×2 confusion matrix rendered once as PNG bytes and reused by both the app and the report, and an fpdf document built around them. I also wrote the k-TSP voting machine in `module4` — top-scoring gene pairs, nine of them by default, each casting a vote — and brought in the eight Tan benchmarks the team profiled against for the rest of the project.

Making the trained model something you keep

The piece I pushed hardest for was **saving and loading a trained model**. There was a bare `pickle.dump` buried in the old `pipeline.py`, but nothing in the app reached it — you built a model, closed the tab, and it was gone. Rebuilding it was minutes of compute and, worse, no guarantee of getting the same thing back.

I argued a fitted model should be an artifact the user owns and carries away, and wrote `model_store` to make that concrete: joblib rather than raw pickle, a JSON sidecar so a saved model records what kind of machine it is and when it was trained, and a `pipeline_to_bytes` that serialises to an in-memory buffer instead of a path — because a browser download needs bytes, not a file sitting on the server.

That last signature is the one that lasted. The merged pipeline's `dumps_machine(machine, metadata) -> bytes` is the same shape: model plus metadata, handed over as bytes. The team later wrapped authenticated encryption around it and gave the format its own extension, but the idea underneath — the model is a portable thing that describes itself — came off this branch.

What actually survived

Not much of the code, and that's the honest part. When the branches merged, `module6_evaluation` became `m6_report`, `model_store` became `ui/model.py`, and my six-module split became `m1 – m6` plus a `ui/` package. The lines were rewritten by other people.

What survived was the *shape*. `ProcessedData(X, y, feature_names)` is now the plain tuple every loader in the merged pipeline returns. That triple is the entire published interface of stage one. I didn't know at the time how much weight it would end up carrying.

zz_imagi
ng

31a4020
Aug 2026
15 files,
+4,424

Testing the seam with something it was never built for

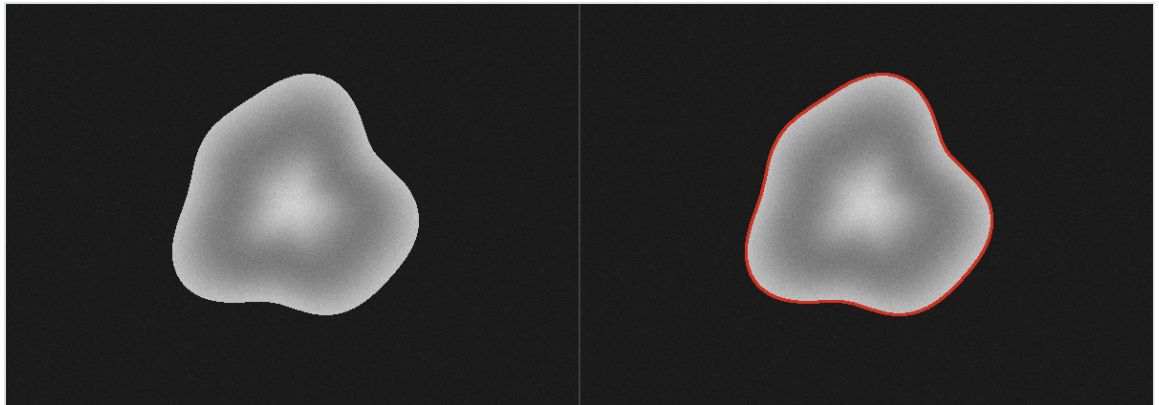
Two months later I picked the project back up to add medical imaging. Everything upstream expected gene expression: a wide float matrix, one row per patient, thousands of columns. Images are not that.

So I set myself a rule before writing anything: **edit no existing pipeline module**. If the contract from the first branch was real, producing that same `(X, y, feature_names)` triple should be enough, and modules 2 through 7 — preprocessing, feature maps, classifiers, cross-validation, the PDF report, the per-prediction explainer — should all work on pixels without knowing anything had changed.

It held. Zero files under `pipeline/` were modified. The imaging support is entirely new files whose output happens to fit.

Getting there meant measuring images rather than learning from them directly: 102 named radiomic features per patient — intensity distribution, Haralick texture from grey-level co-occurrence matrices at two distances, gradient and edge statistics, multi-scale detail energy, shape of the region, and a core-to-rim intensity profile. All numpy and scipy; no torch, no PyRadiomics, no new dependency at all.

Named features were a deliberate choice over deep embeddings. Module 7 explains a prediction by occluding one input at a time and naming what moved it. `im_glc1_con` — fine-scale texture contrast — is something a radiologist can argue with. Embedding dimension 407 is not. There's a bonus I didn't expect: that explainer caps itself at 300 candidates, so at 102 features it stops sampling and becomes *exhaustive*. The "Why?" panel is a complete attribution for imaging models in a way it never was for 16,000 genes.



Left: a synthetic lesion, the kind of thing the module is handed. Right: the region it decides to measure. The outline is computed in your browser by the same rule the module uses – Otsu threshold, then trace the boundary of the largest blob. Everything about shape and core-to-rim only means something if this lands on the right thing.

The part I spent the most care on

Module 5 cross-validates with a bare integer `cv`, which means `StratifiedKFold` with no group awareness. Harmless for gene expression, where one patient is one row. Not harmless here: patients arrive with three scans, or seven. If each became a row, the same patient would sit in the training fold and the test fold at once, and the headline accuracy — the number this whole app exists to report honestly — would be inflated.

I couldn't fix module 5 without breaking my own rule, so I made the mistake unreachable instead. A patient's images are combined into one row *before* `x` exists.

img 1

img 2

img 3

→

1 row · 102 features

Median across a patient's images, not mean – one blurred scan should not drag the row around. Grouping comes from the folder structure, never from parsing filenames.

The stronger move was refusing to offer the alternative. There is no per-image mode and no flag that produces one, because a mode that leaks should not be one option among several — it should not exist. A test asserts the option is absent and that asking for it raises, so the *decision* is guarded, not just today's code. A runtime check confirms the row count equals the number of distinct patients, raised as a real error rather than an `assert`, which `python -O` would strip.

Two things stay out of the feature matrix for the same reason: how many images a patient contributed, and whether the region-finder fell back to a default. Both track the acquisition protocol, which tracks the site, which very often tracks the class. They live in an audit table where a human can look at them.

The problem I didn't see coming

Predicting from a new image only means anything if it's measured the way the training cohort was measured. Three settings change the numbers and can't be recovered from the data — working image size, texture grey levels, and which region gets measured. Get them wrong and nothing errors; you just get a confident answer that quietly means something else.

They had to travel *inside* the model, and I couldn't edit the module that saves models. The way through was a seam already in the codebase: the Build tab copies the dataset's label into the run configuration, and the save step bundles that configuration into the encrypted model file. So the label stopped being a caption and became the transport — `Imaging (radiomics, 256px, 32lv, otsu)`. Thirty-eight characters, ASCII, parsed back on the way in. A model saved today opens on another machine in another session and still knows how to measure a new scan. A test proves a single image reproduces its training row bit-for-bit.

When the report pushed back

My favourite constraint came from the PDF. The prediction report lists the top three features behind each patient, joined into one table cell, and that cell truncates at 48 characters — with the column widths hardcoded in a module I'd sworn not to touch.

Three names plus separators inside 48 characters means roughly 13 characters each. So the naming grammar came out of a layout constraint: `im_<family><param>_<stat>`, capped at 13, with the meaning pushed into a glossary instead of the name. A test measures the longest name in the whole panel and fails if three of them would ever overflow that cell. It felt like a compromise at the time. It's the reason every generated report is still readable.

What I'd tell whoever picks this up

- **The dimensionality regime flipped.** Gene data is ~70 patients by ~16,000 columns. Imaging is ~100 patients by 102. K-TSP and the rank transform both compare feature i against feature j within one patient, which presupposes they share a scale. Circularity against texture entropy does not. They'll run; they're meaningless.
- **Scaling moved from advisable to mandatory.** The features span about six orders of magnitude.
- **The region-finder is the weakest link.** Otsu plus largest blob is right for a lesion on a clean background and wrong for cluttered histology or anything with a burned-in scale bar. That's why the overlay preview and the whole-image escape hatch exist.
- **Ask where the images came from before believing a good score.** JPEG artefacts and native resolution survive normalisation. If one group came off a different scanner, a high AUC means the model found the scanner.
- **My demo cohort scores 1.000, and that means nothing.** I built it to be trivially separable so I could see the plumbing work. It demonstrates that the pipeline runs correctly. It is not a result.

None of that is a reason not to use it. It's the reason the app reports out-of-fold numbers, flags a model that predicts one class for everyone, and shows you the region it measured before you trust what it measured there. Both times I worked on this project, the useful work was making the thing harder to fool yourself with.

Cancer Response Predictor · a seven-module composable ML pipeline with a Streamlit front end · research use only, not a clinical tool.

Branches `zz` (ffd667f) and `zz_imaging` (31a4020).